

Multi-Declaration Z Constructs — Demo

txt2tex

June 13, 2026

Background

The constructs in this file all share a single grammatical shape: a Z paragraph that introduces two or more typed variables in one declaration list, with a conjunction predicate over their projections, optionally followed by a characteristic expression. Until the multi-decl parser fix, none of them parsed natively — users wrote them as raw LaTeX. They now compile to fuzz-clean output.

$[PlayerName, TournamentId, MatchId, VenueName]$

<i>Tournament</i>
$\underline{tournamentId} : TournamentId$
$venue : VenueName$
$tier : \mathbb{N}$
$tier \leq 5$

<i>Player</i>
$\underline{name} : PlayerName$
$ranking : \mathbb{N}$

<i>Match</i>
$\underline{matchId} : MatchId$
$\underline{tournament} : TournamentId$
$winner : PlayerName$
$loser : PlayerName$

1. WYSIWYG line breaks for algebra operators

Long relational-algebra chains can be broken at any operator. Natural newlines work; explicit continuation also works. Display position wraps in array; the chain reads top-to-bottom instead of running off the page.

$\text{Project}_{winner,venue,tier}(\text{Join}(\text{Tournament},$
 $\quad (\text{Project}_{tournament}(\text{Restrict}_{winner='Ali'}(\text{Match})))[\underline{tournamentId}/\underline{tournament}]))$

2. Multi-decl comprehension with conjunction predicate

This is the canonical multi-typed relational-calculus form for any query that crosses two or more relations. Before the fix it required a raw LATEX block; now it parses natively.

$\{ m : Match; t : Tournament; p : Player \mid m.tournament = t.tournamentId \wedge$
 $\quad m.winner = p.name \wedge$
 $\quad t.venue = 'CentreCourt' \bullet$
 $\langle winner == m.winner, venue == t.venue, tier == t.tier \rangle \}$

3. Multi-decl quantifier with conjunction predicate

‘ $\forall s : T; c : U \bullet \text{predicate. body}$ ’ works the same way — nested Quantifier nodes covering the chained declarations.

$$\forall m : \text{Match}; t : \text{Tournament} \mid m.\text{tournament} = t.\text{tournamentId} \wedge t.\text{tier} \leq 2 \bullet m.\text{winner} \neq m.\text{loser}$$

4. Multi-decl quantifier without characteristic expression

When the predicate alone suffices — no ‘ $\bullet$ ’ separator, no body expression — the comprehension defaults the characteristic tuple to the signature (Z RM §3.9).

$$\{ m : \text{Match}; t : \text{Tournament} \mid m.\text{tournament} = t.\text{tournamentId} \wedge t.\text{tier} \leq 2 \}$$

5. Multi-decl mu (definite description)

‘mu’ parses with multi-decl signatures the same way. The current generator emits nested ‘ $(\mu \dots \mid (\mu \dots))$ ’ — *semantically equivalent but not Spivey-canonical. The same special-case treatment lambda just received is a follow-up for mu.Single — declmu (shown here) emits cleanly.*

$$\text{topFinal} == (\mu t : \text{Tournament} \mid t.\text{tier} = 1 \bullet t.\text{venue})$$

6. Multi-decl lambda — Spivey-canonical form

Lambdas of more than one variable now emit as a single λ token with semicolon-joined SchemaText, matching ZRM3.12 and the

$$\text{roster} == (\lambda m : \text{Match}; t : \text{Tournament} \mid m.\text{tournament} = t.\text{tournamentId} \bullet m.\text{winner})$$

7. Paren-wrap fix

Fuzz requires ‘ $(\lambda \dots \bullet \dots)$ ’ — *parentheses around every lambda expression, at any position. The generator previously only wrapped leveled predicates emitted bare ‘ λ ’, which fuzz rejected with Opening parenthesis expected at symbol λ . The fix: drop the ‘paren’*
 $(\lambda x : \mathbb{N} \bullet x + 1)$